

Exam-Like Question

Solution Walkthrough

Sections

1. What do we actually need?
2. The Data Structures
 - Why a Doubly Linked List?
 - Why a Hash Map?
 - The Result
3. Implementation
 - `__init__` and `__len__`
 - `add`
 - `get_cups`
 - `prioritise`
 - `cancel`
 - `serve`
4. Runtime Summary

What do we actually need?

We need a data structure that maintains a **front-to-back order**. But unlike a plain queue, we also need to:

- Jump directly to any customer by *name* (not by reference!) and update or remove them without scanning the whole line.
- Reposition any customer to appear immediately after another without touching everyone else.

Both "jump to by name" and "reposition without a scan" have to run in **$\Theta(1)$ amortised average** time. A single data structure can't satisfy both constraints. For example:

- A linked list would keep order and repositions cheaply, but finding a customer by name requires a $\Theta(n)$ linear scan.
- A hash map gives $\Theta(1)$ lookup, but it has no inherent notion of front-to-back order (since it is *unordered*), so `prioritise` and `serve` are either awkward or expensive.

So, why not both? This is the classic kinda thing you see in exams, so keep that in mind!

Note: The word "queue" in the problem name describes the ordering of customers, not a prescription to use `ArrayQueue`. An `ArrayQueue` only supports $O(1)$ operations at its two ends—it has no way to cancel or reposition an arbitrary element without a full scan. We need something that gives us $O(1)$ access to *any* node once we have a pointer to it.

The Data Structures

Why a Doubly Linked List?

Our `DoublyLinkedList` stores the queue's order. Each node will hold one `(name, cups)` pair, and the sequence of nodes from `header.next` to `trailer.prev` defines the front-to-back ordering of the queue.

The reason why I immediately jumped to a DLL is because of its ability to do **$O(1)$ insertion and deletion**:

- `add_last(val)` appends to the back in $O(1)$.
- `delete_node(node)` unlinks any node in $O(1)$ (if you already have a direct reference to that node). You don't have to find it; you just rewire its neighbours.
- `add_after(node, val)` inserts a new node immediately after any existing node in $O(1)$ —again (as long as you have a direct reference).

This is exactly the surgery that `prioritise` needs: remove a node from wherever it currently sits, and re-insert it somewhere else—all in constant time.

Why a Hash Map?

Ok, so the doubly linked list handles order beautifully, but there's a big problem: its $O(1)$ operations only stay $O(1)$ **if you already have a reference to the node you want**. Finding a node by name on its own would require walking the entire list, which is $O(n)$.

That's where `ChainingHashMap` comes in. It almost acts as an "index":

Key	→	Value

name (string)	→	DLL node (object reference)

Given a customer's name, the hash map lets us retrieve the corresponding *DLL node* in $O(1)$ amortised average time. We can then hand that node directly to the list's operations.

The Result

Combining them eliminates basically each structure's weakness:

Need	Provided by
Maintain front-to-back order	<code>DoublyLinkedList</code>
$O(1)$ insert at back	<code>DoublyLinkedList.add_last</code>
$O(1)$ lookup by name	<code>ChainingHashMap</code>
$O(1)$ reposition / delete any node	<code>DoublyLinkedList.delete_node</code> + <code>add_after</code> (via the hash map pointer)
$O(\text{first})$ ordered traversal from front	<code>DoublyLinkedList.delete_first</code> in a loop

One rule that must never be broken: `self.hash_table[name]` **must always point to the current, live DLL node for that customer**. If a method removes or replaces a node in the list without updating the hash table, future lookups will follow a "stale" reference into a disconnected node.

Implementation

`__init__` and `__len__`

```
def __init__(self):
    self.dll = DoublyLinkedList()
    self.hash_table = ChainingHashTableMap()

def __len__(self):
    return len(self.dll)
```

`DoublyLinkedList.__init__` creates two sentinel nodes—`header` and `trailer`—and wires them together so that `header.next` points to `trailer` and `trailer.prev` points to `header`. These sentinels are never removed and never hold real data; they just mark the two ends of the list. Real nodes are always inserted between them.

`ChainingHashTableMap.__init__` allocates a low-level array of `N` slots (default 64) and fills each slot with an empty `UnsortedArrayMap`. It also creates a fresh MAD hash function bound to that capacity. All key-value pairs will eventually land in one of these slots.

There's nothing else to set up. When both structures are empty, they already agree: the list has no real nodes, and the hash table has no entries.

`__len__` delegates to `self.dll.n`, which `DoublyLinkedList` maintains as a running count and returns directly in $\Theta(1)$. We could equally ask `len(self.hash_table)`, which returns `self.hash_table.n` in the same way—both counts track the number of active orders and will always agree.

`add`

```
def add(self, name, cups):
    if name in self.hash_table:
        node = self.hash_table[name]
        node.data = (name, cups)
    else:
        node = self.dll.add_last((name, cups))
        self.hash_table[name] = node
```

The first line, `name in self.hash_table`, calls `ChainingHashTableMap.__contains__`, which in turn calls `__getitem__` (`val = self[key]`) internally and returns `True` if no `KeyError` is raised. This is $\Theta(1)$ amortised average.

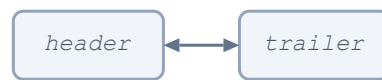
Our two cases are thus:

- **Returning customer.** The spec says we should update the cup count *without changing position*. Because the hash table already holds the node, we retrieve it in $\Theta(1)$ and overwrite `node.data` directly. No list surgery happens at all (`node.prev` and `node.next` are completely untouched) so the customer stays exactly where they were in the queue. The last section of the diagram below shows this: only the highlighted node changes colour (its data changed); everything else—the list structure, the hash table entry, the node object itself—is identical.

- **New customer.** We call `self.dll.add_last((name, cups))`. Under the hood, `add_last` calls `add_after(self.trailer.prev, val)`, which creates a new `Node` object, wires it between the current last real node and the trailer sentinel, increments `self.dll.n`, and **returns the new node**.

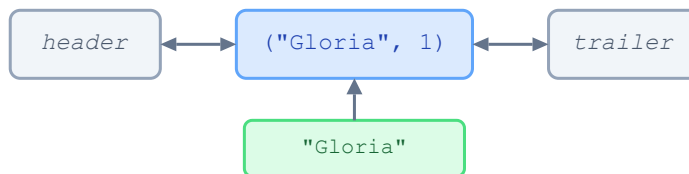
We store that returned node reference in the hash table under `name` by calling `self.hash_table[name] = node`, which is `ChainingHashMap.__setitem__`. That method hashes the key, finds the right bucket, and delegates to `UnsortedArrayMap.__setitem__`—which either updates an existing entry or appends a new `Item` to the bucket's `ArrayList`.

Start



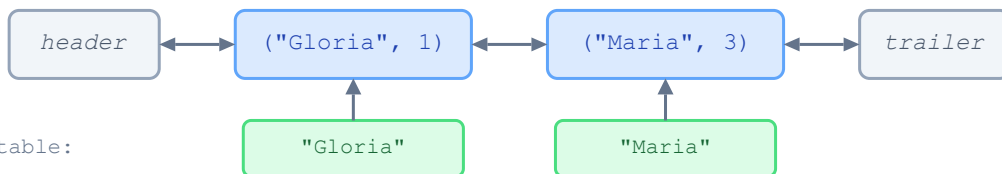
`hash_table: (empty)`

`add("Gloria", 1)`



`hash_table:`

`add("Maria", 3)`



`hash_table:`

`add("Gloria", 2)`

← "Gloria" is already in the queue

Before:



After:



node.data updated in-place · pointers and hash_table entry unchanged

Both branches run in $\Theta(1)$ amortised average time.

get_cups

```

def get_cups(self, name):
    if name not in self.hash_table:
        raise Exception(f"No order found for '{name}'")

    return self.hash_table[name].data[1]
  
```

The guard check (`name not in self.hash_table`) runs the same `__contains__` path described in `add`— $\Theta(1)$ amortised average. If `name` isn't found, we raise immediately as was specified, rather than letting Python

produce a confusing `KeyError` from deep inside the hash table.

If the name is present, `self.hash_table[name]` retrieves the DLL node: the MAD function computes the bucket index, and the `UnsortedArrayMap` at that slot does a short linear scan to find the matching entry and return its value (the node object). We then access `.data`, which is the `(name, cups)` tuple stored on that node, and return index `1`—the cup count (index `0` would give us back the name, which we already have).

```
get_cups("Maria")
```

DLL: [header] ↔ [("Gloria", 1)] ↔ [("Maria", 3)] ↔ [("Vincy", 2)] ↔ [trailer]

```
hash_table:  "Gloria"  _____  "Maria"  _____  "Vincy"  _____
                                                    ↑
                                                    |
                                                    .data = ("Maria", 3)
                                                    .data[1] = 3   ← returned
```

prioritise

```
def prioritise(self, name, prev_name):
    if name not in self.hash_table:
        raise Exception(f"No order found for '{name}'")
    if prev_name not in self.hash_table:
        raise Exception(f"No order found for '{prev_name}'")

    data = self.dll.delete_node(self.hash_table[name])
    new_node = self.dll.add_after(self.hash_table[prev_name], data)
    self.hash_table[name] = new_node
```

Both guard checks use `__contains__` as before. They're kept separate so that if only one name is missing, the exception message can say which one—rather than a generic failure.

After the guards pass, we have two direct node references in hand:

1. `self.hash_table[name]`—the node we want to move.
2. `self.hash_table[prev_name]`—the node we want to move it after.

Let's trace through `prioritise("Maria", "Vincy")` on the queue below:

```
Start: prioritise("Maria", "Vincy")
```

DLL: [header] ↔ [("Gloria", 2)] ↔ [("Maria", 3)] ↔ [("Vincy", 2)] ↔ [trailer]

```
hash_table:  "Gloria"  _____  "Maria"  _____  "Vincy"  _____
                                                    ↑
                                                    ↑
                                                    ↑
```

Step 1: remove the node from its current position.

`self.dll.delete_node(self.hash_table[name])` reaches into the list and rewires the surrounding nodes: it sets `node.prev.next = node.next` and `node.next.prev = node.prev`, effectively stitching the gap closed. It then calls `node.disconnect()`, which nulls out the node's `data`, `prev`, and `next` fields, and decrements `self.dll.n`. The node object still exists in memory but it's no longer part of any list. `delete_node` returns the data it saved before disconnecting—the `(name, cups)` tuple—which we store in `data`.

After `delete_node(hash_table["Maria"])`

```

DLL:  [header] ↔ [("Gloria", 2)] ↔ [{"Vincy", 2}] ↔ [trailer]
      ↑               ↑
hash_table: "Gloria" —| "Maria" —x "Vincy" —|
              ↑
              stale—points to
              the disconnected node

data = ("Maria", 3) ← saved by delete_node before disconnecting

```

Step 2: re-insert it after `prev_name`.

`self.dll.add_after(self.hash_table[prev_name], data)` creates a brand new `Node` holding that same `data`, wires it between `prev_name`'s node and whatever came after it, increments `self.dll.n`, and returns the new node.

After `add_after(hash_table["Vincy"], ("Maria", 3))`

```

DLL:  [header] ↔ [("Gloria", 2)] ↔ [{"Vincy", 2}] ↔ [{"Maria", 3}]
↔ [trailer]
      ↑               ↑               ↑
hash_table: "Gloria" —| "Maria" —x "Vincy" —| new_node |
              ↑               ↑
              still stale

```

Step 3: update the hash table.

`self.hash_table[name] = new_node` is non-negotiable. The old node was disconnected and is now garbage. If we skipped this line, the hash table would still map `name` to that dead object, and any future call to `add`, `get_cups`, `cancel`, or `prioritise` on this customer would silently operate on a node that isn't in the list.

After `hash_table["Maria"] = new_node`

```

DLL:  [header] ↔ [("Gloria", 2)] ↔ [{"Vincy", 2}] ↔ [{"Maria", 3}] ↔
[trailer]
      ↑               ↑               ↑
hash_table: "Gloria" —| "Vincy" —| "Maria" —|

```

cancel

`cancel` does the same two-step cleanup as the removal in `prioritise`, just without the re-insertion.

The order of these two lines matters: we must retrieve the node from the hash table *before* deleting its entry. If we deleted the hash table entry first, the node reference would be gone and we'd have no way to tell the list which node to unlink.

serve

```
def serve(self, first=1):
    count = min(first, len(self.dll))
    result = []

    for _ in range(count):
        name, cups = self.dll.delete_first()
        del self.hash_table[name]
        result.append((name, cups))

    return result
```

`serve` is the only method allowed to be slower than $\Theta(1)$, and deliberately so—returning `first` items necessarily takes $\Theta(\text{first})$ time just to build the result list.

`min(first, len(self.dll))` handles the case where `first` $\geq$ the current queue length: rather than running off the end of the list, we cap `count` at however many orders actually exist and drain the whole thing.

Each iteration does three things:

1. `self.dll.delete_first()` calls `delete_node(self.header.next)`—the node immediately after the header sentinel, which is always the front of the queue. `delete_node` stitches the header directly to what was the second node, calls `node.disconnect()` to null out the removed node's fields, decrements `self.dll.n`, and returns the stored `(name, cups)` tuple.
2. `del self.hash_table[name]` takes the name we just unpacked from that tuple and removes it from the hash table, exactly as in `cancel`. This keeps the two structures in sync—a node that no longer exists in the list should not have an entry in the hash table.
3. `result.append((name, cups))` adds the tuple to the output list.

Here's the full picture for `serve(2)` on the post-`prioritise` queue:

```
serve(2) → count = min(2, 3) = 2
```

Start:

```
DLL:    [header] ↔ [("Gloria", 2)] ↔ [("Vincy", 2)] ↔ [("Maria", 3)] ↔
[trailer]
result: []
```

Iteration 1—`delete_first()` returns `("Gloria", 2)`:

```
DLL:    [header] ↔ [("Vincy", 2)] ↔ [("Maria", 3)] ↔ [trailer]
result: [("Gloria", 2)]
```

Iteration 2—`delete_first()` returns `("Vincy", 2)`:

```
DLL:    [header] ↔ [("Maria", 3)] ↔ [trailer]
result: [("Gloria", 2), ("Vincy", 2)]
```



```
Returned: [("Gloria", 2), ("Vincy", 2)]
```

Since each of those three steps is $\Theta(1)$ amortised average, and we repeat them `count` times, the full loop runs in $\Theta(\text{count}) = \Theta(\text{first})$.

Runtime Summary

Operation	Time Complexity	Reasoning
<code>__len__</code>	$\Theta(1)$	Reads <code>self.dll.n</code> directly
<code>add</code>	$\Theta(1)$ amortised avg.	Hash map lookup + DLL <code>add_last</code> or in-place node update
<code>get_cups</code>	$\Theta(1)$ amortised avg.	Hash map lookup, then tuple index
<code>prioritise</code>	$\Theta(1)$ amortised avg.	Two hash map lookups + DLL <code>delete_node</code> + <code>add_after</code>
<code>cancel</code>	$\Theta(1)$ amortised avg.	Hash map lookup + DLL <code>delete_node</code> + hash map delete
<code>serve(first)</code>	$\Theta(\text{first})$	Loop of <code>first</code> $\times$ (DLL <code>delete_first</code> + hash map delete)

The "amortised average" qualifier comes from the hash map: individual operations can be $\Theta(n)$ in the pathological worst case (all keys collide into one chain), but with a good hash function and dynamic resizing, a sequence of `n` operations completes in $\Theta(n)$ average time.